PROJECT REPORT

Customer Churn Prediction

A Machine Learning System for Predicting Customer Attrition

Table of Contents

1. Executive Summary

This report documents the end-to-end development of a customer churn prediction system, from raw data through a deployed, interactive web application. The objective was to build a model that flags customers likely to cancel their service, so that retention teams can intervene proactively rather than reactively.

Using a dataset of 1,000 customer records, I explored churn patterns, engineered a compact feature set, trained and tuned five candidate machine learning algorithms, and selected a linear Support Vector Machine (SVM) as the production model based on cross-validated accuracy. The model, together with its feature scaler, has been packaged and wired into a Streamlit application that allows a non-technical user to enter a customer's age, gender, tenure, and monthly charges and receive an instant churn prediction.

- **Headline result:** the final model reaches 90.5% accuracy on a held-out test set, matching the best-performing alternative (Logistic Regression) while offering a simpler, more stable decision boundary.
- **Important caveat:** the dataset is heavily imbalanced (88.3% of customers are labeled as churned), so accuracy alone overstates real-world performance. This is addressed in Section 10 and should be resolved before the model is used for high-stakes retention decisions.

2. Business Objective

Customer churn — the loss of a paying customer — is one of the most expensive problems a subscription or service-based business faces, since acquiring a new customer typically costs far more than retaining an existing one. The goal of this project was to answer a single operational question for the business:

> *"Given basic information we already hold about a customer, can we predict — before they leave — whether they are at risk of churning?"*

A reliable early-warning signal allows customer success and marketing teams to prioritize outreach, targeted offers, or support interventions toward the customers most likely to leave, rather than spreading retention budget evenly across the whole base.

3. Dataset Overview

The source file customer_churn_data.csv contains 1,000 customer records across 10 columns, combining demographic, account, and service attributes with the churn outcome label.

Column	Type	Description
CustomerID	Integer	Unique identifier for each customer record
Age	Integer	Customer age in years (observed range: 12–83)
Gender	Categorical	Male / Female
Tenure	Integer	Months the customer has held their account (0–122)
MonthlyCharges	Float	Recurring monthly bill amount ($30.00–$119.96)
ContractType	Categorical	Month-to-Month / One-Year / Two-Year

Column	Type	Description
InternetService	Categorical	DSL / Fiber Optic (297 records missing)
TotalCharges	Float	Cumulative amount billed to date
TechSupport	Categorical	Whether the customer subscribes to tech support (Yes/No)
Churn	Categorical (Target)	Whether the customer churned — Yes / No

- **Class balance:** 883 customers (88.3%) are labeled Churn = Yes and 117 (11.7%) are labeled Churn = No. This is a materially imbalanced dataset — see Section 10.
- **Missing data:** InternetService was missing for 297 of 1,000 records (29.7%); all other columns were complete. No duplicate rows were found.

4. Data Preparation

Before modeling, the raw data was inspected and cleaned in the analysis notebook (notebook.ipynb):

- **Missing values:** the 297 missing InternetService entries were filled with an empty placeholder value rather than dropped, preserving all 1,000 records for analysis.
- **Duplicate check:** the dataset was screened for duplicate rows; none were found.
- **Target & feature scope:** of the 9 available predictor columns, four were carried forward into the model — Age, Gender, Tenure, and MonthlyCharges — selected for their direct availability at the point a churn score is needed and their simplicity for a first production model. ContractType, InternetService, TechSupport, and TotalCharges were explored during EDA but not included in the final feature set (see Section 10 for the impact of this decision).
- **Encoding:** Gender was label-encoded (Female = 1, Male = 0) and the target Churn was label-encoded (Yes = 1, No = 0) to make both compatible with scikit-learn estimators.

5. Exploratory Data Analysis — Key Findings

Exploratory analysis in the notebook examined the distribution of each variable and its relationship to churn using group-by summaries, a correlation matrix, and histograms.

Metric	Churn = No	Churn = Yes
Average Monthly Charges	$62.55	$75.96
Average Tenure (months)	30.3	17.5
Average Age (years)	43.5	44.8

- **Tenure is the strongest visible churn signal:** customers who churned had an average tenure of just 17.5 months, versus 30.3 months for retained customers — newer customers churn at a noticeably higher rate.
- **Price sensitivity:** churned customers paid, on average, about $13 more per month ($75.96 vs. $62.55), consistent with higher bills increasing the likelihood of cancellation.
- **Age was not a strong differentiator:** average age was nearly identical between churned (44.8) and retained (43.5) customers.
- **Contract type:** 511 customers are on Month-to-Month contracts, 289 on One-Year, and 200 on Two-Year contracts; average monthly charges decrease slightly with longer commitment terms ($75.91 → $73.82 → $71.33), suggesting longer contracts may carry a modest loyalty discount.
- **Correlation matrix:** TotalCharges is strongly correlated with Tenure (r = 0.89), as expected since total billing accumulates over the customer's lifetime. No other numeric features showed strong linear correlation with each other, reducing multicollinearity concerns for the chosen feature set.

6. Feature Engineering

The modeling feature set was finalized as:

- Age (numeric)
- Gender (binary: 1 = Female, 0 = Male)
- Tenure (numeric, months)
- MonthlyCharges (numeric, USD)

The data was split into training and test sets using an 80/20 split (sklearn.model_selection.train_test_split). All four features were then standardized with scikit-learn's StandardScaler (zero mean, unit variance) so that no single feature — particularly MonthlyCharges, which has a much larger numeric range than Gender — would dominate distance- or margin-based algorithms. The fitted scaler was exported as scaler.pkl for reuse at inference time.

7. Model Development & Comparison

Five classification algorithms were trained and evaluated on the same 80/20 train-test split, with hyperparameters for four of them tuned via 5-fold GridSearchCV. Model quality was measured using accuracy_score against the held-out test set.

Algorithm	Tuning Approach	Test Accuracy
Logistic Regression	Baseline, default parameters	90.5%
Support Vector Machine (SVM)	GridSearchCV — C: [0.01, 0.1, 0.5, 1], kernel: [linear, rbf, poly]. Best: C=0.01, kernel=linear	90.5%
K-Nearest Neighbors	GridSearchCV — n_neighbors: [3,5,7,9], weights: [uniform, distance]. Best: n_neighbors=5, weights=distance	89.5%
Random Forest	GridSearchCV — n_estimators: [32,64,128,256], max_features, bootstrap. Best: n_estimators=128, max_features=sqrt, bootstrap=True	89.5%
Decision Tree	GridSearchCV over criterion, splitter, max_depth, min_samples_split/leaf	84.0%

Logistic Regression and the tuned linear SVM tied for the top score at 90.5% accuracy. K-Nearest Neighbors and Random Forest followed closely at 89.5%, while the Decision Tree classifier under-performed the rest at 84.0%, consistent with a single tree's tendency to overfit on a modest, four-feature dataset.

8. Final Model Selection

The production model is a Support Vector Classifier with a linear kernel and regularization strength C = 0.01 (sklearn.svm.SVC(C=0.01, kernel='linear')), selected as the GridSearchCV best_estimator_. It was chosen over the equally-accurate Logistic Regression baseline because:

- Its strong regularization (low C) favors a wide, simple decision margin, which tends to generalize better on a small dataset (1,000 records) than a less-constrained boundary.
- A linear kernel keeps the decision boundary interpretable — churn risk increases smoothly with the weighted combination of input features, avoiding the risk of an overly complex, hard-to-explain boundary.
- It matched every other candidate on measured accuracy, so the simpler, more stable option was preferred.

The trained model was serialized with joblib and exported as model.pkl. The paired StandardScaler (scaler.pkl) must always be applied to raw inputs before they are passed to the model, since the model was trained exclusively on standardized features.

- **Inference contract:** the model expects a feature vector in the exact order [Age, Gender, Tenure, MonthlyCharges], scaled by scaler.pkl, and returns 1 for a predicted churn and 0 for a predicted retention.

9. Deployment — Streamlit Application

To make the model usable by non-technical staff, I built a lightweight Streamlit web application (app.py) that wraps the trained model behind a simple form:

- The user enters four values through native Streamlit widgets: Age (number input, 18–100), Tenure in months (number input, 0–130), Monthly Charges (number input, minimum $30), and Gender (dropdown: Male / Female).
- On clicking “Predict Churn,” the app assembles the four inputs in the required order, encodes Gender to match training (Female = 1, Male = 0), applies the saved scaler.transform(), and passes the result to the model's .predict() method.
- The result is displayed instantly as either “Churn” or “Not Churn,” with a celebratory balloon animation for engagement.

This gives customer success or sales teams a self-serve tool: they can look up or estimate a customer's basic profile and get an immediate risk read, without needing to run any code or open the notebook.

- **Deployment package:** app.py (Streamlit UI and inference logic), model.pkl (trained SVM), and scaler.pkl (fitted StandardScaler). All three files must be deployed together and kept in sync — retraining the model requires re-exporting both model.pkl and scaler.pkl as a matched pair.

12. Conclusion

This project delivered a working, end-to-end churn prediction pipeline: cleaned and explored data, five tuned candidate models, a selected linear SVM classifier, and a deployed Streamlit application that turns that model into a tool anyone on the team can use. The application is functional today and gives stakeholders a tangible way to interact with the model's predictions.